

EXP 26

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(

    n_samples=600,

    n_features=2,

    n_informative=2,

    n_redundant=0,

    n_classes=3,

    n_clusters_per_class=1,

    random_state=42

)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = MLPClassifier(

    hidden_layer_sizes=(10,),

    activation='tanh',

    solver='adam',

    max_iter=1000,

    random_state=42

)

model.fit(X_train, y_train)
```

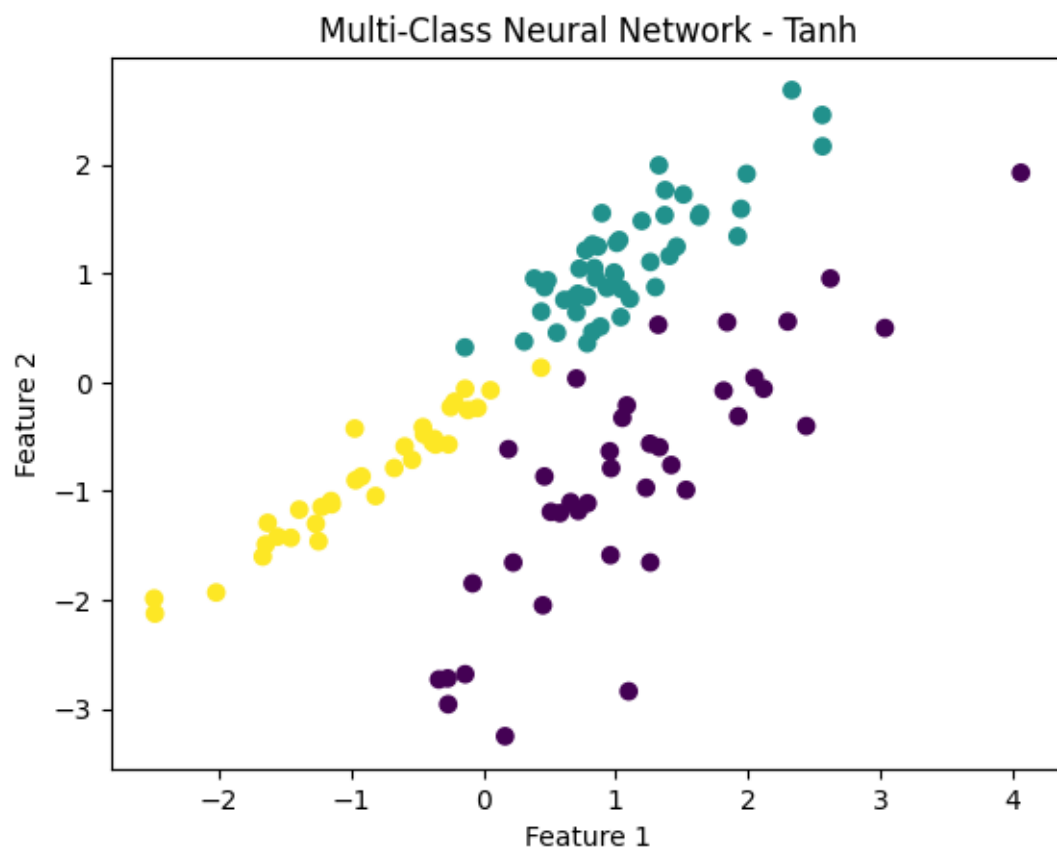
```
y_pred = model.predict(X_test)
print("Activation Function: Tanh")
print("Number of Classes: 3")
print("Test Accuracy:", accuracy_score(y_test, y_pred))
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis')
plt.title("Multi-Class Neural Network - Tanh")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

Output:

Activation Function: Tanh

Number of Classes: 3

Test Accuracy: 0.925



EXP 27

Code:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(

    n_samples=500,

    n_features=2,

    n_informative=2,

    n_redundant=0,

    n_classes=2,

    random_state=42

)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = MLPClassifier(

    hidden_layer_sizes=(10,),

    activation='relu',

    learning_rate_init=0.001,

    max_iter=1000,

    random_state=42

)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Activation Function: ReLU")  
  
print("Learning Rate: 0.001")  
  
print("Number of Classes: 2")  
  
print("Test Accuracy:", accuracy_score(y_test, y_pred))  
  
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='coolwarm')  
  
plt.title("Two-Class Neural Network - ReLU")  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

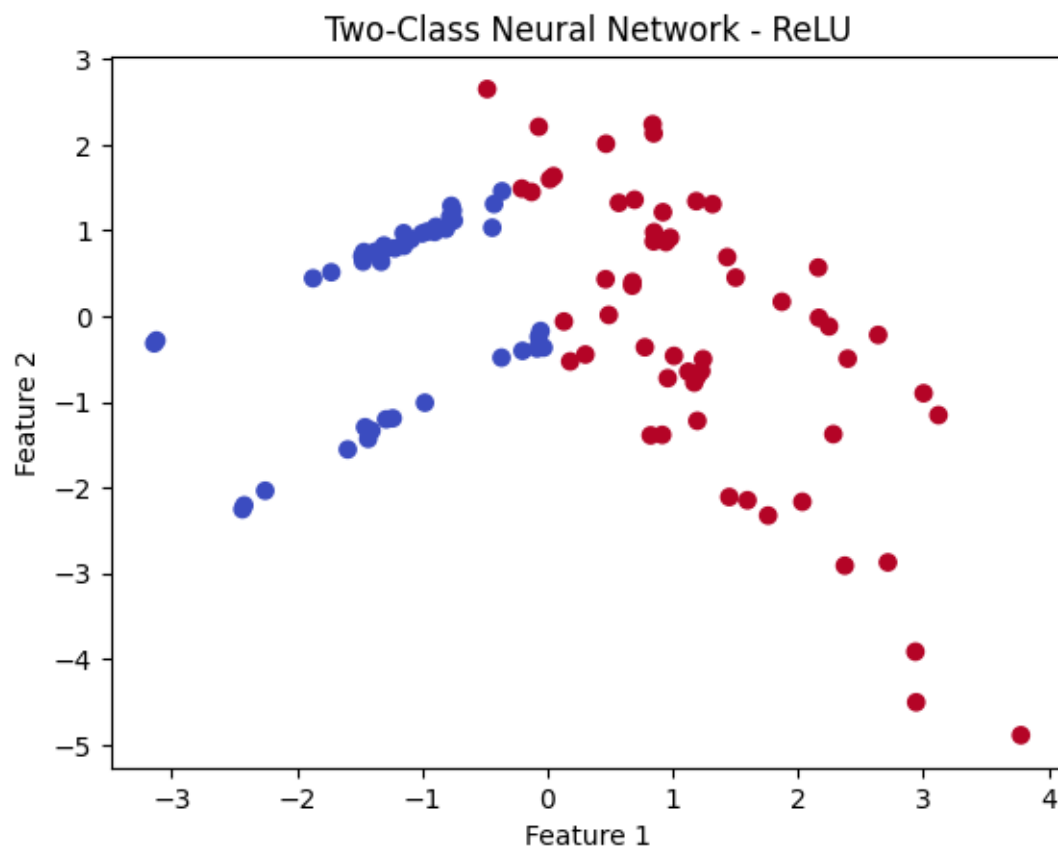
Output:

Activation Function: ReLU

Learning Rate: 0.001

Number of Classes: 2

Test Accuracy: 0.88



EXP 28

CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=600,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    learning_rate_init=0.001,
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Activation Function: ReLU")  
  
print("Learning Rate: 0.001")  
  
print("Number of Classes: 3")  
  
print("Test Accuracy:", accuracy_score(y_test, y_pred))  
  
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis')  
  
plt.title("Multi-Class Neural Network - ReLU, LR=0.001")  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

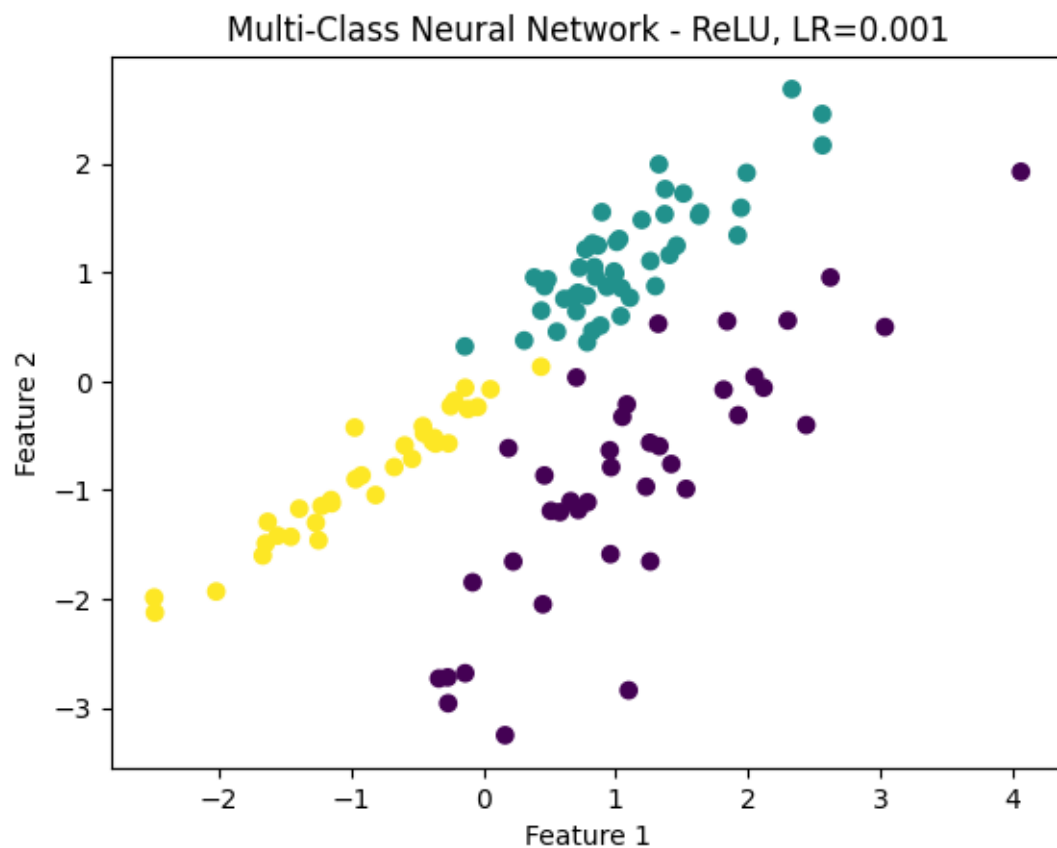
OUTPUT

Activation Function: ReLU

Learning Rate: 0.001

Number of Classes: 3

Test Accuracy: 0.925



EXP 29

CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=600,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_classes=3,
    n_clusters_per_class=1,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(3,),
    activation='tanh',
    learning_rate_init=0.001,
    max_iter=1500,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Activation Function: Tanh")  
  
print("Hidden Neurons: 3")  
  
print("Number of Classes: 3")  
  
print("Test Accuracy:", accuracy_score(y_test, y_pred))  
  
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis')  
  
plt.title("Multi-Class - Tanh - 3 Hidden Neurons")  
  
plt.xlabel("Feature 1")  
  
plt.ylabel("Feature 2")  
  
plt.show()
```

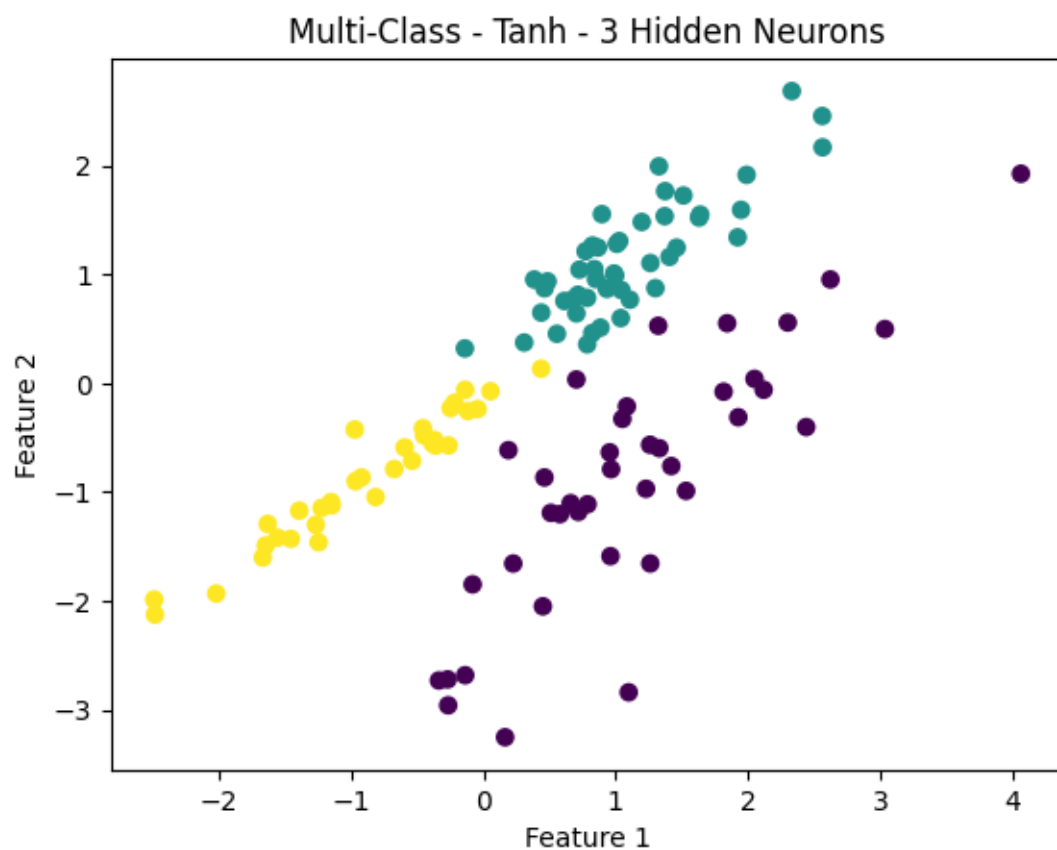
OUTPUT

Activation Function: Tanh

Hidden Neurons: 3

Number of Classes: 3

Test Accuracy: 0.925



EXP 30

CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(

    n_samples=500,

    n_features=2,

    n_informative=2,

    n_redundant=0,

    n_classes=2,

    random_state=42

)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = MLPClassifier(

    hidden_layer_sizes=(3,),

    activation='tanh',

    learning_rate_init=0.001,

    max_iter=1500,

    random_state=42

)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Activation Function: Tanh")
```

```
print("Hidden Neurons: 3")  
print("Number of Classes: 2")  
print("Test Accuracy:", accuracy_score(y_test, y_pred))  
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='coolwarm')  
plt.title("Two-Class - Tanh - 3 Hidden Neurons")  
plt.xlabel("Feature 1")  
plt.ylabel("Feature 2")  
plt.show()
```

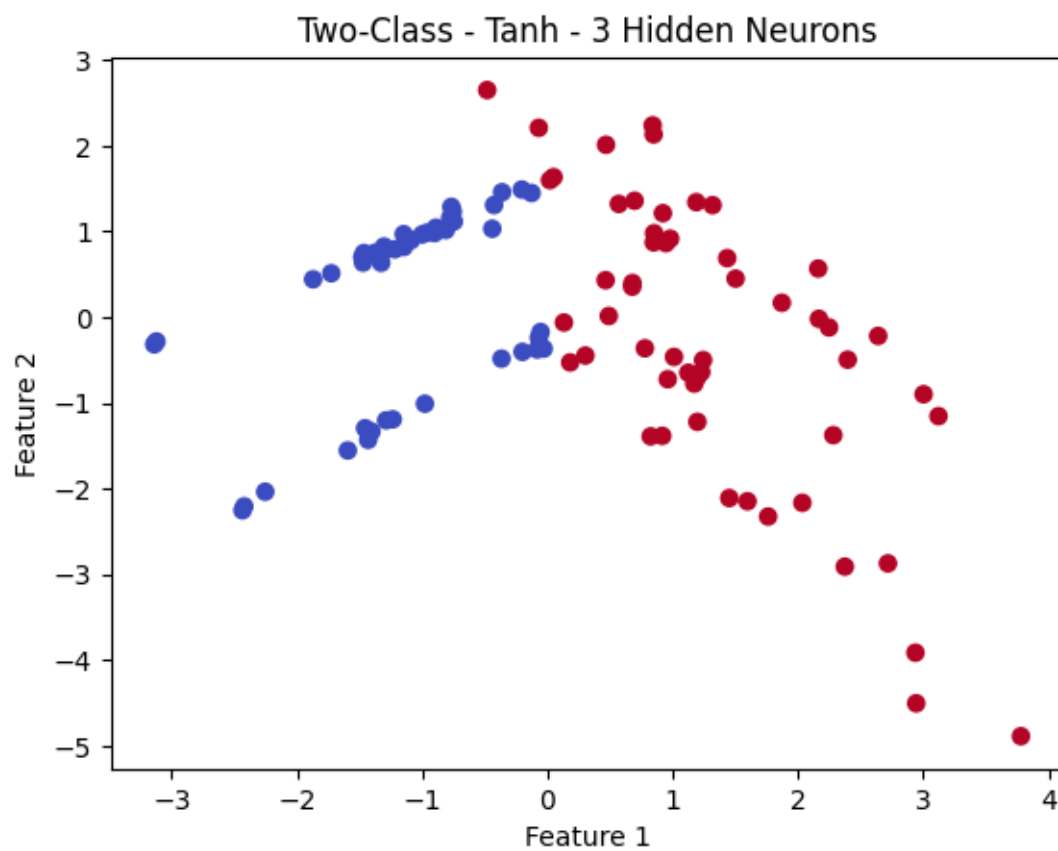
OUTPUT

Activation Function: Tanh

Hidden Neurons: 3

Number of Classes: 2

Test Accuracy: 0.9



EXP 31

CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_classification

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_classification(

    n_samples=600,

    n_features=2,

    n_informative=2,

    n_redundant=0,

    n_classes=3,

    n_clusters_per_class=1,

    random_state=42

)

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)

model = MLPClassifier(

    hidden_layer_sizes=(10,),

    activation='relu',

    learning_rate_init=0.03,

    max_iter=1000,

    random_state=42

)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Activation Function: ReLU")

print("Learning Rate: 0.03")

print("Number of Classes: 3")

print("Test Accuracy:", accuracy_score(y_test, y_pred))

plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='viridis')

plt.title("Multi-Class - ReLU - Learning Rate 0.03")

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.show()
```

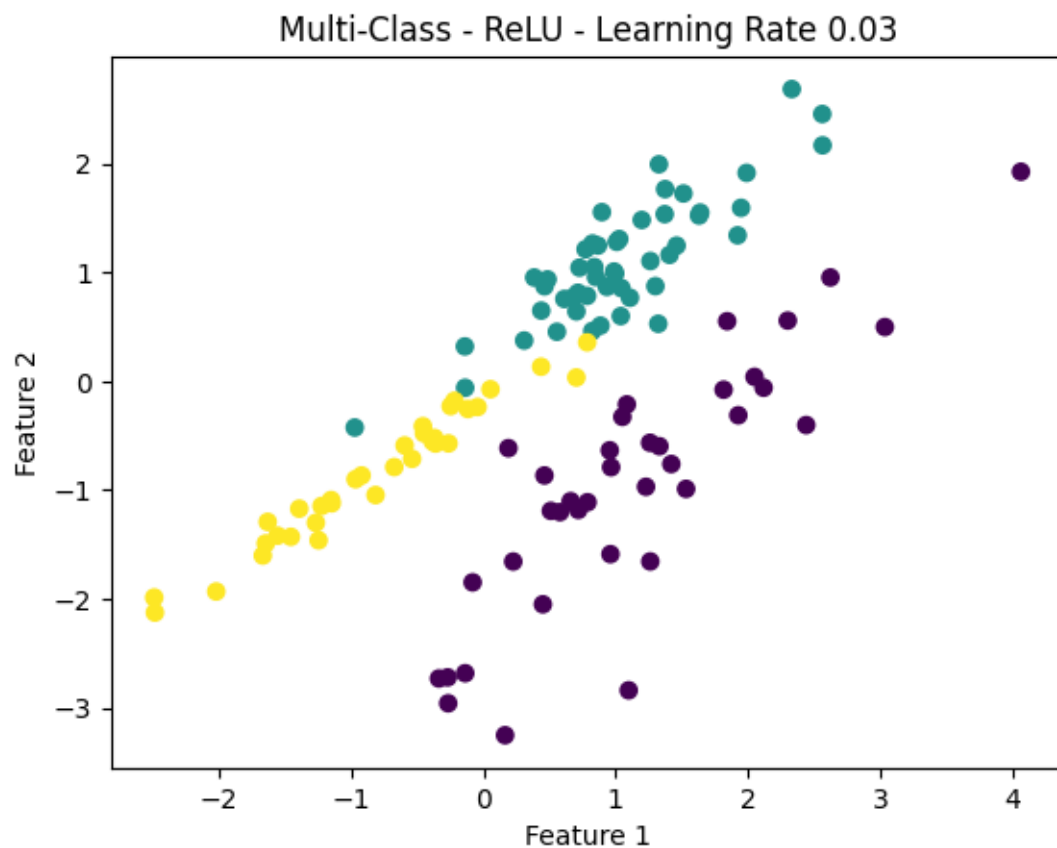
OUTPUT

Activation Function: ReLU

Learning Rate: 0.03

Number of Classes: 3

Test Accuracy: 0.9416666666666667



EXP 32

CODE

```
import matplotlib.pyplot as plt

from sklearn.datasets import make_circles

from sklearn.model_selection import train_test_split

from sklearn.neural_network import MLPClassifier

from sklearn.metrics import accuracy_score

X, y = make_circles(
    n_samples=600,
    noise=0.08,
    factor=0.5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(10, 10),
    activation='tanh',
    learning_rate_init=0.001,
    max_iter=2000,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Dataset: Two Circular Data")

print("Activation Function: Tanh")

print("Test Accuracy:", accuracy_score(y_test, y_pred))
```

```
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_pred, cmap='coolwarm')  
plt.title("Two Circular Data - Tanh Activation")  
plt.xlabel("Feature 1")  
plt.ylabel("Feature 2")  
plt.show()
```

OUTPUT

Dataset: Two Circular Data

Activation Function: Tanh

Test Accuracy: 1.0

